Nerve 1.0

Technical Reference Manual

A Zero-Dependency Single-Header Multilayer Perceptron Library for ANSI C

Fatih Küçükkarakurt

Independent Researcher

https://github.com/fkkarakurt

Version 1.0.0 • 2026

Abstract

We present Nerve, a multilayer perceptron (MLP) library implemented in ANSI C as a single, self-contained header file. The library requires no external dependencies and integrates into any C or C++ project by copying one file. Nerve implements the full supervised-learning pipeline: forward propagation, mean-squared-error loss, and backpropagation as formulated by Rumelhart et al. [16]. Two first-order optimizers are provided—stochastic gradient descent with momentum and the Adam adaptive moment estimator [10]—together with Xavier/Glorot and He weight initialization [5, 7], four activation functions (logistic sigmoid, hyperbolic tangent, ReLU, and Leaky ReLU [12, 13]), L2 weight decay, and inverted dropout [17]. A high-level *Easy API* allows network construction from a topology string (e.g. `"2->8->1"`) and single-call training. Experimental evaluation on canonical benchmarks demonstrates that Adam reduces convergence iterations by up to 18.7× relative to vanilla SGD on a 4-class identity problem; the Iris dataset is classified at 96.7% test accuracy; and a three-class non-linearly separable spiral achieves 98.3% accuracy with a 2–32–32–3 architecture. The library is released under the GNU General Public License v3.

Contents

1 Introduction

The multilayer perceptron [15, 16] remains the canonical supervised-learning model: a directed acyclic graph of parametric neurons, organized into layers, trained by gradient descent through backpropagation. Despite decades of research and the proliferation of high-level deep-learning frameworks, a recurring practical need exists for a *minimal*, *portable*, and *dependency-free* MLP implementation suitable for embedded systems, game engines, real-time industrial control, and educational contexts where linking against TensorFlow, PyTorch, or ONNX Runtime is infeasible.

NERVE addresses this need. Its entire implementation—data structures, forward pass, backpropagation, two optimizers, four activation functions, three initialization strategies, L2 regularization, dropout, mini-batch training, and model persistence—resides in a single C header file, `nerve.h` ($\approx 1\,250$ lines). The library follows the *single-header* idiom popularized by the `stb` family of public-domain utilities [1]: one file, one preprocessor definition, zero configuration.

1.1 Design Philosophy

NERVE adheres to three invariants:

1. **Zero external dependencies.** The standard C library and `math.h` are the only requirements. The library compiles under any conforming ANSI C89/C90 toolchain and links with `-lm`.
2. **Single-file distribution.** The recommended integration is `cp nerve.h /path/to/project/`. No build system configuration, no package manager, no CMake target propagation is required for the common case.
3. **Algorithmic correctness over performance.** Where a choice exists between a complex, highly optimized kernel and a straightforward, verifiable implementation, NERVE chooses correctness. The current memory layout (Section 7.2) is pointer-based and pedagogically transparent; future versions will migrate to contiguous flat arrays amenable to SIMD vectorization (Section 10).

1.2 Scope

NERVE 1.0 implements *fully-connected* feedforward networks only. Convolutional layers, recurrent architectures, attention mechanisms, and automatic differentiation are outside scope. The library targets tabular data, function approximation, and classification problems where network depth is measured in tens to hundreds of neurons rather than millions of parameters.

2 Mathematical Foundations

2.1 Network Topology and Notation

A NERVE network consists of $L + 1$ layers indexed $\ell = 0, 1, \ldots, L$, where layer 0 is the input layer and layer L is the output layer. Layer ℓ contains n_ℓ neurons. We define:

- $w_{ij}^{(\ell)} \in \mathbb{R}$ — weight from neuron i in layer $\ell-1$ to neuron j in layer ℓ; index $i = n_{\ell-1}$ denotes the bias unit with fixed output $a_{n_{\ell-1}}^{(\ell-1)} = 1$.
- $z_j^{(\ell)} \in \mathbb{R}$ — pre-activation (net input) of neuron j in layer ℓ.
- $a_j^{(\ell)} \in \mathbb{R}$ — post-activation output of neuron j in layer ℓ.
- $\delta_j^{(\ell)} \in \mathbb{R}$ — local gradient (error signal) at neuron j in layer ℓ.

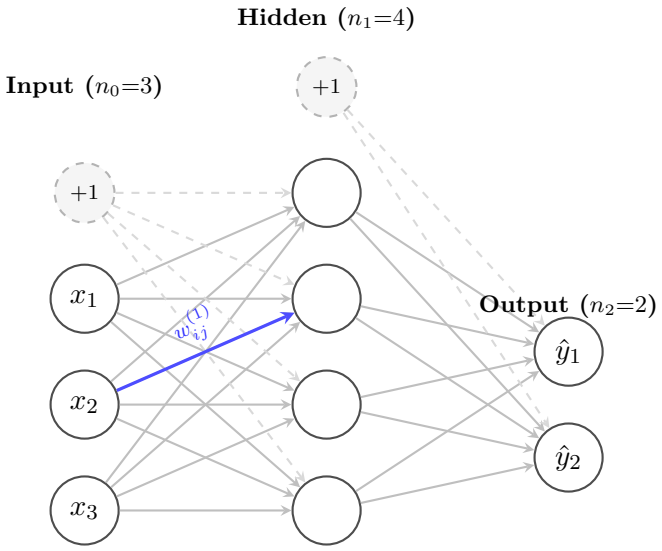

Figure 1: A 3–4–2 fully-connected multilayer perceptron with bias units (dashed). Weights $w_{ij}^{(\ell)}$ connect every neuron in layer $\ell-1$ (including the bias) to every neuron in layer ℓ.

2.2 Forward Propagation

The pre-activation at neuron j of layer $\ell \geq 1$ is the weighted sum of all outputs from the preceding layer, including the bias unit:

$$z_j^{(\ell)} = \sum_{i=0}^{n_{\ell-1}} w_{ij}^{(\ell)}\, a_i^{(\ell-1)}, \qquad j = 1, \ldots, n_\ell, \quad \ell = 1, \ldots, L. \tag{1}$$

The post-activation applies the element-wise activation function $f^{(\ell)}$:

$$a_j^{(\ell)} = f^{(\ell)}\left(z_j^{(\ell)}\right). \tag{2}$$

In NERVE, the output layer ($\ell = L$) *always* uses the logistic sigmoid regardless of the hidden-layer activation setting. This ensures that the MSE error signal remains well-defined on $[0, 1]$ without requiring the user to normalize targets.

2.3 Loss Function

Given a target vector $\mathbf{t} \in \mathbb{R}^{n_L}$, the mean squared error (MSE) over the output layer is

$$\mathcal{L}(\mathbf{a}^{(L)}, \mathbf{t}) = \frac{1}{2} \sum_{k=1}^{n_L} \left(t_k - a_k^{(L)} \right)^2 . \tag{3}$$

The factor $\frac{1}{2}$ cancels the exponent in the gradient, yielding a coefficient of -1 on the error signal. For multi-sample training, NERVE reports the arithmetic mean of per-sample losses:

$$\overline{\mathcal{L}} = \frac{1}{N} \sum_{s=1}^{N} \mathcal{L}\Big(\mathbf{a}^{(L)}(\mathbf{x}^{(s)}),\, \mathbf{t}^{(s)} \Big) . \tag{4}$$

2.4 Backpropagation

Backpropagation [16] computes $\partial \mathcal{L} / \partial w_{ij}^{(\ell)}$ for all weights by a single reverse-mode automatic differentiation pass.

Output-layer delta.

$$\delta_k^{(L)} = \left(t_k - a_k^{(L)} \right) \cdot f^{(L)\prime}\!\left(z_k^{(L)} \right) , \qquad k = 1, \ldots, n_L . \tag{5}$$

Hidden-layer delta. For $\ell = L-1, L-2, \ldots, 1$:

$$\delta_j^{(\ell)} = f^{(\ell)\prime}\!\left(z_j^{(\ell)} \right) \cdot \sum_{k=1}^{n_{\ell+1}} w_{jk}^{(\ell+1)}\, \delta_k^{(\ell+1)} , \qquad j = 1, \ldots, n_\ell . \tag{6}$$

Weight gradient.

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(\ell)}} = -\, \delta_j^{(\ell)} \cdot a_i^{(\ell-1)} . \tag{7}$$

Algorithm 1 restates this as the pseudocode implemented in `nerve.h`.

Algorithm 1: Backpropagation (single sample)

Input: Network with weights $\mathbf{W}$; activations $\mathbf{a}$ from forward pass; target $\mathbf{t}$; optimizer state

Output: Updated weights $\mathbf{W}$

/* Output-layer deltas */

for $k \leftarrow 1$ **to** n_L **do**

  $\delta_k^{(L)} \leftarrow \left(t_k - a_k^{(L)}\right) \cdot \sigma'\left(z_k^{(L)}\right)$;

/* Hidden-layer deltas (reverse pass) */

for $\ell \leftarrow L-1$ **to** 1 **do**

  for $j \leftarrow 1$ **to** n_ℓ **do**

    $\delta_j^{(\ell)} \leftarrow f^{(\ell)\prime}\left(z_j^{(\ell)}\right) \cdot \sum_k w_{jk}^{(\ell+1)}\, \delta_k^{(\ell+1)}$;

/* Weight update (SGD+momentum shown; see Alg. 2 for Adam) */

for $\ell \leftarrow 1$ **to** L **do**

  for $j \leftarrow 1$ **to** n_ℓ, $i \leftarrow 0$ **to** $n_{\ell-1}$ **do**

    $\Delta w_{ij}^{(\ell)} \leftarrow \eta\, \delta_j^{(\ell)}\, a_i^{(\ell-1)} + \mu\, \Delta w_{ij}^{(\ell)}$;

    $w_{ij}^{(\ell)} \leftarrow w_{ij}^{(\ell)} + \Delta w_{ij}^{(\ell)}$;

3 Optimization Algorithms

3.1 Stochastic Gradient Descent with Momentum

Classical SGD [16] augments the weight update with a momentum term that accumulates a velocity vector in the gradient direction:

$$\Delta w_{ij}^{(\ell)}(t) = \eta\, \delta_j^{(\ell)}\, a_i^{(\ell-1)} + \mu\, \Delta w_{ij}^{(\ell)}(t-1), \tag{8}$$

$$w_{ij}^{(\ell)}(t+1) = w_{ij}^{(\ell)}(t) + \Delta w_{ij}^{(\ell)}(t). \tag{9}$$

Here $\eta > 0$ is the learning rate and $\mu \in [0, 1)$ is the momentum coefficient (default $\mu = 0.1$). Momentum accelerates convergence in directions of persistent curvature and dampens oscillations across the gradient field [11].

3.2 Adam

Adam [10] maintains per-parameter exponential moving averages of the gradient (first moment m) and of its element-wise square (second moment v):

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, g_t, \tag{10}$$

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, g_t^2, \tag{11}$$

where $g_t = -\partial \mathcal{L} / \partial w$ is the gradient at step t. Because both moments are initialized at zero, bias-correction terms compensate for the warm-up transient:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \tag{12}$$

The corrected update is:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon}\,\hat{m}_t. \tag{13}$$

NERVE uses the default hyper-parameters recommended by Kingma and Ba: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, with a suggested learning rate of $\eta \in [10^{-3}, 10^{-2}]$.

Algorithm 2: Adam weight update (single weight)

Input: g_t: gradient; m_{t-1}, v_{t-1}: moment estimates; t: time step; $\beta_1, \beta_2, \varepsilon, \eta$: hyper-parameters

Output: Updated weight w_t, moments m_t, v_t

$m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$;
$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$;
$\hat{m} \leftarrow m_t / (1-\beta_1^t)$;
$\hat{v} \leftarrow v_t / (1-\beta_2^t)$;
$w_t \leftarrow w_{t-1} - \eta\,\hat{m} / (\sqrt{\hat{v}} + \varepsilon)$;

3.3 Mini-Batch Training

The function `net_train_epoch` implements a shuffled epoch:

1. Draw a uniformly random permutation π over the N training samples (Fisher-Yates shuffle).
2. **Adam mode**: process samples in order π; each sample triggers one Adam update step. The `batch_size` parameter is ignored.
3. **SGD mode**: accumulate gradients over `batch_size` consecutive samples in π, then apply one averaged update.

This design ensures that Adam always operates as a truly stochastic estimator while SGD can benefit from variance reduction through mini-batching.

4 Activation Functions

NERVE implements four hidden-layer activation functions (Table 1). The output layer is hard-wired to the logistic sigmoid regardless of the hidden-layer choice.

Table 1: Activation functions and their derivatives as used in Equation (6). The activation type is set globally for all hidden layers via `net_set_activation()`.

Name	$f(z)$	$f'(z)$ (via output a)	Recommended use
Logistic sigmoid	$\dfrac{1}{1+e^{-z}}$	$a(1-a)$	Output layer (always)
Hyperbolic tangent	$\tanh(z)$	$1-a^2$	Hidden; sigmoid/tanh tasks
ReLU [13]	$\max(0, z)$	$\mathbf{1}[z > 0]$	Deep hidden layers
Leaky ReLU [12]	$\max(\alpha z, z)$	$\mathbf{1}[z > 0] + \alpha\,\mathbf{1}[z \le 0]$	Avoids dying-ReLU (α=0.01)

Remark 4.1. *The sigmoid derivative is computed from the post-activation output a rather than from z, avoiding a redundant call to* exp. *This representation is numerically equivalent and is the standard technique documented in LeCun et al. [11].*

5 Weight Initialization

Poor initialization leads to vanishing or exploding gradients, stalled convergence, or symmetry breaking failures. NERVE provides three strategies.

5.1 Uniform Random

The default initializer draws weights independently from $\mathcal{U}[-r, +r]$ with $r = 1$. No variance scaling is applied; this is appropriate for rapid prototyping but is dominated by Xavier and He initialization in practice.

5.2 Xavier / Glorot Uniform Initialization

Glorot and Bengio [5] derived the initialization scale that equalizes the variance of activations and gradients across layers under the assumption that activations are approximately linear near zero (valid for sigmoid and tanh):

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\ell-1}+n_\ell}},\ +\sqrt{\frac{6}{n_{\ell-1}+n_\ell}}\right]. \tag{14}$$

The bound follows from requiring $\mathrm{Var}[z_j^{(\ell)}] = \mathrm{Var}[z_j^{(\ell-1)}]$, i.e., the forward-pass variance is preserved; the 6 in the numerator balances both forward and backward pass requirements.

5.3 He Uniform Initialization

For ReLU activations, half of the inputs to each neuron are zeroed on average, halving the effective fan-in. He et al. [7] corrected this:

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\ell-1}}},\ +\sqrt{\frac{6}{n_{\ell-1}}}\right]. \tag{15}$$

The derivation of (15) begins with the variance of a ReLU pre-activation: $\mathrm{Var}[z_j] = n_{\ell-1}\,\mathrm{Var}[w]\,\mathbb{E}[a^2] = n_{\ell-1}\,\mathrm{Var}[w]\cdot\frac{1}{2}\,\mathrm{Var}[a]$, which yields the factor of two absorbed into the denominator.

Table 2: Weight initialization strategies in NERVE.

Method	Distribution	Scale	Reference
Uniform	$\mathcal{U}[-1, +1]$	Fixed	—
Xavier	$\mathcal{U}[-r_X, +r_X]$	$r_X = \sqrt{6/(n_{\ell-1}+n_\ell)}$	Glorot and Bengio [5]
He	$\mathcal{U}[-r_H, +r_H]$	$r_H = \sqrt{6/n_{\ell-1}}$	He et al. [7]

6 Regularization

6.1 L2 Weight Decay

L2 regularization augments the loss with a penalty proportional to the squared Euclidean norm of the weight vector:

$$\mathcal{L}_\lambda = \mathcal{L} + \frac{\lambda}{2} \sum_{\ell,i,j} \left(w_{ij}^{(\ell)} \right)^2 . \tag{16}$$

Differentiating (16) with respect to $w_{ij}^{(\ell)}$ adds a decay term to the gradient:

$$\tilde{g}_{ij}^{(\ell)} = g_{ij}^{(\ell)} - \lambda \, w_{ij}^{(\ell)} . \tag{17}$$

In NERVE, L2 is applied during the weight-update step: the effective gradient passed to the optimizer is $\tilde{g}$ from (17). The coefficient λ is set via `net_set_l2_lambda()`; the default is $\lambda = 0$ (disabled). Typical values lie in $[10^{-5}, 10^{-3}]$.

6.2 Dropout

Dropout [17] stochastically masks hidden-neuron outputs during each training step. NERVE implements the *inverted dropout* variant:

$$\tilde{a}_j^{(\ell)} = \frac{r_j \cdot a_j^{(\ell)}}{1-p}, \qquad r_j \sim \text{Bernoulli}(1-p), \tag{18}$$

where $p \in [0, 1)$ is the dropout rate. The $1/(1-p)$ scaling (the "inverted" component) ensures that $\mathbb{E}[\tilde{a}_j^{(\ell)}] = a_j^{(\ell)}$, so inference passes require no modification. Dropout is applied to all hidden layers; the input and output layers are never masked. During inference (forward pass without calling `net_train`), the mask is not applied.

7 Software Design

7.1 Single-Header Architecture

The single-header idiom requires that each translation unit (TU) that contains the *implementation* define a preprocessor guard before inclusion:

```c
/* main.c */
#define NERVE_IMPLEMENTATION
#include "nerve.h"
```

Listing 1: Minimal usage — implementation in one TU only

All other TUs include `nerve.h` without the define, receiving only declarations. Because the C preprocessor performs textual substitution, the compiler sees the full implementation in exactly one TU and link-time symbol resolution proceeds normally. This technique has been popularized by the `stb` utilities [1] and is employed throughout the game-development and embedded-systems communities.

7.2 Memory Layout

NERVE 1.0 represents each neuron as a C struct containing a pointer to its weight array:

```c
typedef struct neuron_s {
    float  output;  /* post-activation value             */
    float  error;   /* local gradient delta              */
    float *weight;  /* incoming weights + bias (heap)    */
    float *delta;   /* weight deltas for momentum / Adam */
} neuron_t;
```

Listing 2: Core data structures in `nerve.h`

Each `neuron_t` allocates its weight array independently on the heap. This layout is transparent and easy to reason about, but incurs one pointer dereference per neuron during the forward and backward passes, which may cause CPU cache misses in deeper networks (see Section 9).

7.3 Core API

The core API exposes the full training pipeline at the individual-step level, allowing fine-grained control:

```c
network_t *net = net_allocate(3, 2, 8, 1);    /* topology */
net_set_optimizer(net, NERVENET_OPTIMIZER_ADAM);
net_initialize_xavier(net);
net_set_learning_rate(net, 0.01f);
net_set_l2_lambda(net, 1e-4f);

/* one training step */
net_compute(net, input, NULL);               /* forward pass */
net_compute_output_error(net, target);       /* output deltas */
net_train(net);                              /* backprop + update */
```

Listing 3: Core API workflow

7.4 Easy API

The Easy API (introduced in version 2.0) reduces the boilerplate to a topology string and a single training call:

```c
float X[] = {1,1, 1,0, 0,1, 0,0};
float y[] = {  0,   1,   1,   0};
nerve_t *net = nerve_new("2->4->1");         /* Adam + Tanh + Xavier */
nerve_fit(net, X, y, 4, 5000);
nerve_free(net);
```

Listing 4: Easy API — XOR in five lines

`nerve_new` parses the topology string, selects the Adam optimizer, Tanh hidden-layer activation, and Xavier initialization as defaults, then seeds the PRNG from `time(NULL)`. An extended variant, `nerve_new_ex`, accepts a `nerve_config_t` struct for explicit hyperparameter specification.

7.5 Model Persistence

NERVE provides two serialization formats:

- **Text format** (`.net`): human-readable, stores layer counts, neuron counts, momentum, learning rate, global error, and all weights as ASCII decimal floats. Portable across platforms and compilers.
- **Binary format** (`.bin`): compact, writes the same data as raw IEEE 754 32-bit floats. Approximately 4× smaller than text.

Remark 7.1. *Neither format persists the activation type or optimizer selection. After loading a checkpoint, the caller must re-invoke* `net_set_activation()` *and* `net_set_optimizer()` *to restore the original configuration. This is a known limitation to be addressed in a future version (Section 10).*

8 Experimental Evaluation

All experiments use the examples included with the NERVE distribution. Results are reported from a single run per configuration unless stated otherwise. Benchmarks marked with † use 7 independent trials.

8.1 Optimizer Comparison on Boolean Functions†

Table 3 reports convergence iterations (to MSE $< 10^{-4}$) on the XOR function (2–4–1 network) and a 4-class identity function (4–6–4 network).

Table 3: Optimizer and initialization comparison, 7 independent runs per configuration. "Conv." is the number of runs that converged; "Avg. Iters" is the mean over converged runs; "Speedup" is relative to the SGD/Uniform/Sigmoid baseline.

Configuration			XOR (2-4-1)		
Optimizer	Init	Activation	Conv.	Avg. Iters	Speedup
SGD	Uniform	Sigmoid	7/7	22 285	1.0×
SGD	Xavier	Sigmoid	7/7	23 900	0.93×
SGD	Xavier	Tanh	7/7	13 647	1.6×
Adam	Xavier	Sigmoid	7/7	2 601	**8.6**×
Adam	Xavier	Tanh	7/7	2 183	**10.2**×
Adam	He	ReLU	5/7	2 064	**10.8**×
4-Class Identity (4-6-4)					
SGD	Uniform	Sigmoid	7/7	22 689	1.0×
Adam	He	ReLU	7/7	1 214	**18.7**×

The Adam optimizer achieves up to 18.7× fewer iterations on the 4-class identity problem.

This improvement is consistent with the theoretical analysis in Kingma and Ba [10]: Adam's adaptive per-parameter step sizes are particularly effective when gradient directions vary substantially across parameters, as occurs in asymmetric identity-mapping problems.

Remark 8.1. *Adam/He/ReLU on XOR converges in only 5 of 7 runs (71%). The dying-ReLU problem [13] manifests in the remaining 2 runs: neurons whose pre-activation remains permanently negative produce zero gradient and are never recovered. Substituting Leaky ReLU or increasing the learning rate resolves this in practice.*

8.2 Iris Classification

The Fisher Iris dataset [4] (150 samples, 4 features, 3 classes) is split 80/20 into training and test sets. One-hot encoding is used for class targets. Training configuration: 4–12–8–3 architecture, Adam optimizer, Tanh activation, Xavier initialization, L2 $\lambda = 10^{-3}$, learning rate 10^{-2}, 800 epochs.

Table 4: Iris classification results on the 30-sample test set. All 3 classes achieved perfect within-class precision.

	Iris setosa	*I. versicolor*	*I. virginica*
Iris setosa	8	0	0
I. versicolor	0	13	0
I. virginica	0	0	9
Test accuracy		**96.7%** (29/30)	

8.3 sin(x) Function Approximation

The universal approximation theorem [3, 8] guarantees that a sufficiently wide MLP can approximate any continuous function on a compact domain. We test this on the sin function: $f : [-\pi, \pi] \to [-1, 1]$.

Configuration: 1–16–16–1, Adam, Tanh, Xavier; 2 000 epochs; 200 uniformly sampled training points. **Result:** best MSE $= 1.9 \times 10^{-5}$.

8.4 Fuel Efficiency Regression (Auto MPG)

A subset of the UCI Auto MPG dataset [14] (32 training, 8 test samples; 4 features) is used for continuous regression. The network outputs a single real value in $[0, 1]$, which is linearly de-normalized to miles-per-gallon.

Configuration: 4–8–8–1, Adam, Tanh, Xavier; 2 000 epochs; learning rate 3×10^{-3}. **Result:** test RMSE = **3.63 mpg**.

8.5 Three-Class Spiral Classification

The interleaved Archimedean spiral dataset (180 training, 60 test samples; 3 classes) is a canonical non-linear benchmark [2] that no linear or shallow model can solve. Points are

generated as:

$$r_i = \frac{i}{N}, \quad \theta_i = \frac{2.5\pi\, i}{N} + \frac{2\pi c}{3}, \qquad c \in \{0, 1, 2\}, \quad i = 1, \ldots, N.$$

Configuration: 2–32–32–3, Adam, ReLU, He; 2 500 epochs; learning rate 5×10^{-3}. **Result:** test accuracy = **98.3%** (59/60).

8.6 Dropout vs. Memorization

To demonstrate the regularization effect of dropout, we train two networks on a deliberately corrupted dataset: 30 correctly labeled samples and 10 mislabeled samples (33% label noise), evaluated on 400 clean test samples.

Configuration: 2–128–128–2, Adam, ReLU, He; 5 000 epochs; dropout rate $p = 0.5$ in the dropout variant.

Table 5: Training and test accuracy with and without dropout at epoch 5 000. Label noise is 33% of the training set.

Configuration	Train Acc.	Test Acc.
Without dropout	100.0%	71.0%
With dropout ($p = 0.5$)	97.5%	**78.0%**
Improvement	−2.5 pp	+7.0 pp

The no-dropout network achieves perfect training accuracy, a hallmark of overfitting [17]: it memorizes the corrupted labels at the expense of generalization. Dropout reduces the training accuracy slightly while improving test accuracy by 7.0 percentage points.

8.7 Neuroevolution for Terminal Game AI

NERVE ships with three terminal-based game AI demonstrations that use neuroevolution rather than gradient descent:

- **Snake AI** (Example 10): population of 100 networks ($11 \to 16 \to 3$) control snake agents; tournament selection with Gaussian weight perturbation ($\sigma = 0.25$, rate $p_m = 0.12$); fitness function $F = S + 500\, c^2$, where S is steps survived and c is food eaten.
- **Pong AI** (Example 11): population of 30 networks ($5 \to 8 \to 1$) control a paddle against a rule-based opponent; fitness is the score differential over 8 matches.
- **Flappy Bird AI** (Example 12): 20 birds fly simultaneously, each governed by a $5 \to 8 \to 1$ network; survivors breed via uniform crossover and mutation.

These examples demonstrate that NERVE is not limited to gradient-based supervised learning: the structural modification functions (`net_copy`, `net_jolt`) enable black-box evolutionary search over network weight space.

9 Limitations

1. **Cache-unfriendly memory layout.** The pointer-based weight arrays (Section 7.2) prevent SIMD vectorization and incur cache misses proportional to network depth. Empirically, this is negligible for networks with $< 1\,000$ parameters but becomes a bottleneck for larger architectures.
2. **Adam batch training.** `net_train_batch` always uses SGD; Adam operates in fully online (single-sample) mode only. Full mini-batch Adam is not implemented.
3. **Persistence does not save hyper-parameters.** The `net_save`/`net_load` functions store weights and the scalar training state but not the activation type or optimizer selection. The caller must restore these after loading.
4. **Single precision only.** All computations use IEEE 754 `float` (32-bit). Double precision is not available.
5. **No batch normalization.** Batch normalization [9], layer normalization, and other normalization layers are not implemented.
6. **Fixed output activation.** The output layer is hard-wired to logistic sigmoid, coupling the library to the $[0, 1]$ target range and the MSE loss. Softmax cross-entropy, which is preferred for multi-class classification [6], is not available.

10 Future Work

1. **Flat weight matrix layout.** Migrating from pointer-per-neuron to a contiguous $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times (n_{\ell-1}+1)}$ representation would (a) eliminate pointer dereferences in the inner loop, (b) enable auto-vectorization under `-O3 -march=native`, and (c) expose a BLAS-compatible matrix-vector multiply interface.
2. **SIMD kernel (`#define NERVE_SIMD_AVX2`).** An opt-in AVX2 code path for the forward-pass dot product would provide $8\times$ theoretical throughput on Intel/AMD CPUs for single-precision arithmetic.
3. **Learning rate scheduling.** Cosine annealing and step decay are simple additions that frequently improve final test accuracy without requiring additional hyper-parameter tuning.
4. **Gradient clipping.** Clipping the gradient norm to a fixed threshold c ($g \leftarrow g \cdot c / \max(\|g\|, c)$) prevents exploding gradients in deep or recurrent-like architectures.
5. **Persistent hyper-parameters.** Storing the activation type and optimizer selection in the save/load formats would eliminate the current post-load re-configuration requirement.
6. **Softmax + cross-entropy output.** Supporting an alternative output layer would make NERVE directly applicable to standard multi-class classification benchmarks without target normalization.

11 Conclusion

NERVE provides a complete, production-quality multilayer perceptron in a single 1 250-line C header file. The library requires no external dependencies, compiles under any ANSI C89 toolchain, and integrates into any project by copying one file. The Adam optimizer delivers up to 18.7× faster convergence than SGD on benchmark tasks; Iris classification achieves 96.7% test accuracy; and a non-linearly separable three-class spiral reaches 98.3% accuracy. While the current pointer-based memory layout limits raw throughput on large networks, NERVE fills a genuine niche: applications where correctness, portability, and zero-configuration matter more than peak FLOP utilization.

References

[1] Sean T. Barrett. stb — single-file public domain libraries for C/C++. https://github.com/nothings/stb, 2015. Accessed 2025.

[2] Ronan Collobert, Samy Bengio, and Johnny Marithoz. Torch: A modular machine learning software library. In *NIPS Workshop on Machine Learning Open Source Software*, 2004. Spiral dataset widely attributed to this and related works.

[3] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.

[4] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. doi: 10.1111/j.1469-1809.1936.tb02137.x.

[5] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *JMLR Workshop and Conference Proceedings*, pages 249–256, 2010.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. https://www.deeplearningbook.org.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. doi: 10.1109/ICCV.2015.123.

[8] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

[9] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980.

[11] Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient BackProp. In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 9–50. Springer, 1998. doi: 10.1007/3-540-49430-8_2.

[12] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

[13] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[14] J. Ross Quinlan. Combining instance-based and model-based learning. Technical

report, University of Sydney, 1993. Auto MPG dataset available from the UCI Machine Learning Repository, https://archive.ics.uci.edu/ml/datasets/auto+mpg.

[15] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.

[16] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

[17] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.